

Doc. no.: P0602R1

Date: 2017-04-30

Audience: Library Evolution Working Group

Reply-to: Zhihao Yuan <zy at miator dot net>

variant and optional should propagate copy/move triviality

Rationale

Making variant and optional conditionally trivially copy constructible, trivially move constructible, trivially copy assignable, and trivially move assignable significantly improves codegen and performance when they are nested or being put into containers.

Given the spirit of the resolution of [LWG 2900](#) and feedback from implementers, this paper now proposes a full-fledged approach that presents in libc++ and a future update in MS STL, that is to make the four operations individually trivial if all alternatives have the corresponding operations being trivial. libstd++ implementation is [on the way](#).

This change comes with a (good) potential ABI breakage in user's code – a small enough trivially copyable optional & variant took or returned by a function can be passed via register after applying the change, therefore we propose to accept this change as a defect report against C++17.

Wording

This wording is relative to N4660.

Modify 23.6.3.1 [optional.ctor] as follows:

```
constexpr optional(const optional& rhs);
```

[...]

Remarks: This constructor shall be defined as deleted unless `is_copy_constructible_v<T>` is true. If `is_trivially_copy_constructible_v<T>` is true, this constructor shall be a constexpr **trivial** constructor.

```
constexpr optional(optional&& rhs) noexcept(see below );
```

[...]

Remarks: The expression inside `noexcept` is equivalent to `is_nothrow_move_constructible_v<T>`. This constructor shall not participate in overload resolution unless `is_move_constructible_v<T>` is true. If `is_trivially_move_constructible_v<T>` is true, this constructor shall be a constexpr **trivial** constructor.

Modify 23.6.3.3 [optional.assign] as follows:

```
optional<T>& operator=(const optional& rhs);
```

[...]

Remarks: If any exception is thrown, [...]. This operator shall be defined as deleted unless `is_copy_constructible_v<T>` is true and `is_copy_assignable_v<T>` is true. **If `is_trivially_copy_constructible_v<T>` && `is_trivially_copy_assignable_v<T>` && `is_trivially_destructible_v<T>` is true, this assignment operator shall be trivial.**

```
optional<T>& operator=(optional&& rhs) noexcept(see below );
```

[...]

Remarks: The expression inside `noexcept` is equivalent to: [...] This operator shall not participate in overload resolution unless `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true. **If `is_trivially_move_constructible_v<T>` && `is_trivially_move_assignable_v<T>` && `is_trivially_destructible_v<T>` is true, this assignment operator shall be trivial.**

Modify 23.7.3.1 [variant.ctor] as follows:

```
variant(const variant& w);
```

[...]

Remarks: This ~~function shall not participate in overload resolution~~ constructor shall be defined as deleted unless `is_copy_constructible_v<Ti>` is true for all *i*. If `is_trivially_copy_constructible_v<Ti>` is true for all *i*, this constructor shall be trivial.

```
variant(variant&& w) noexcept(see below );
```

[...]

Remarks: The expression inside `noexcept` is equivalent to [...]. This function shall not participate in overload resolution unless `is_move_constructible_v<Ti>` is true for all *i*. If `is_trivially_move_constructible_v<Ti>` is true for all *i*, this constructor shall be trivial.

Modify 23.7.3.3 [variant.assign] as follows:

```
variant& operator=(const variant& rhs);
```

[...]

Remarks: This ~~function shall not participate in overload resolution~~ operator shall be defined as deleted unless `is_copy_constructible_v<Ti> && is_copy_assignable_v<Ti>` is true for all *i*. If `is_trivially_copy_constructible_v<Ti> && is_trivially_copy_assignable_v<Ti> && is_trivially_destructible_v<Ti>` is true for all *i*, this assignment operator shall be trivial. [...]

```
variant& operator=(variant&& rhs) noexcept(see below );
```

[...]

Remarks: This function shall not participate in overload resolution unless `is_move_constructible_v<Ti> && is_move_assignable_v<Ti>` is true for all *i*. If `is_trivially_move_constructible_v<Ti> && is_trivially_move_assignable_v<Ti> && is_trivially_destructible_v<Ti>` is true for all *i*, this assignment operator shall be trivial. The expression inside `noexcept` is equivalent to: [...]

Acknowledgments

Thanks Casey Carter for evolving the implementations and explaining the results.